UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

EXPERIMENT 5

Student Name: Jatin Tasoria **UID: 25MCC20054**

Branch: MCA-Cloud Computing **Section/Group: MCC 101-A**

Semester: 2nd **Date of Performance: 01/03/26**

Subject Name: Technical Training-I **Subject Code: 25CAP-652**

1. **Aim**

   To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic.

2. **Objective:**

   After completing this practical, the learner will be able to:

   - Understand sequential row-by-row data processing using cursors
   - Learn the lifecycle of a cursor: DECLARE, OPEN, FETCH, CLOSE
   - Perform row-level manipulation using procedural logic
   - Handle cursor-related exceptions and termination conditions
   - Apply cursor logic in real-world enterprise scenarios

3. **Practical / Experiment Steps**

   a. Start the system.
   b. Open pgAdmin.
   c. Create and select the database in which you want to perform the experiment.
   d. Establish connection to the database.
      Execute cursor-based queries

4. Procedure of the Practical

1. Start the system and log in
2. Start the PostgreSQL service
3. Open PostgreSQL client (psql or pgAdmin)
4. Create a new database
5. Create the required table
6. Insert sample data into the table
7. Declare and use CURSOR
8. Execute queries successfully
9. Save the work for documentation

5. SQL Queries Used in the Experiment

a. Simple Forward-Only Cursor

```
DO $$
DECLARE
   rec RECORD;
   emp_cursor CURSOR FOR
      SELECT id, name, salary FROM employee;
BEGIN
   OPEN emp_cursor;
   LOOP
      FETCH emp_cursor INTO rec;
      EXIT WHEN NOT FOUND;
      RAISE NOTICE 'ID: %, Name: %, Salary: %', rec.id, rec.name,
rec.salary;
   END LOOP;
   CLOSE emp_cursor;
END $$;
```

b. Salary Update

```
DO $$
DECLARE
   rec RECORD;
   emp_cursor CURSOR FOR
```

```
        SELECT id, experience, salary FROM employee;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO rec;
        EXIT WHEN NOT FOUND;

        IF rec.experience > 5 THEN
            UPDATE employee
            SET salary = salary * 1.20
            WHERE id = rec.id;
        ELSE
            UPDATE employee
            SET salary = salary * 1.10
            WHERE id = rec.id;
        END IF;
    END LOOP;
    CLOSE emp_cursor;
END $$;
```

c. Exception Handling with Cursor

```
DO $$
DECLARE
    rec RECORD;
    emp_cursor CURSOR FOR SELECT id FROM employee;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO rec;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Employee ID: %', rec.id;
    END LOOP;
    CLOSE emp_cursor;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'An error occurred while processing cursor.';
END $$;
```

6. Input / Output Analysis (I/O Analysis)

- Simple Forward:

```
NOTICE:  ID: 1, Name: Rahul, Salary: 30000.00
NOTICE:  ID: 2, Name: Amit, Salary: 25000.00
NOTICE:  ID: 3, Name: Priya, Salary: 40000.00
NOTICE:  ID: 4, Name: Sneha, Salary: 22000.00
NOTICE:  ID: 5, Name: Arjun, Salary: 28000.00
DO
```

- Salary Update using Cursor:

```
DO

Query returned successfully in 42 msec.
```

- Exceptional Handling:

```
NOTICE:  Employee ID: 1
NOTICE:  Employee ID: 2
NOTICE:  Employee ID: 3
NOTICE:  Employee ID: 4
NOTICE:  Employee ID: 5
DO
```

7. Learning Outcome

- Ability to implement and manage cursors
- Understanding of row-by-row processing
- Knowledge of exception handling in PL/pgSQL
- Application of cursor logic in payroll and enterprise systems